

Programmation Orientée Objet en Python

IPNET Institute of Technology

*Classes, Encapsulation, Polymorphisme,
Héritage et Bonnes Pratiques*

15 mars 2026

Table des matières

0.1	Introduction et Rappels sur Python	4
0.1.1	Pourquoi la POO ?	4
0.1.2	Rappel : Types de Base en Python	4
0.1.3	Rappel : Fonctions	4
0.1.4	Rappel : Portée des Variables	4
0.1.5	Transition : Limites de la Programmation Procédurale	4
0.2	Concepts de Base de la POO	5
0.2.1	Classe vs Objet	5
0.2.2	Exemple : Classe Voiture	5
0.2.3	Diagramme UML : Classe Voiture	5
0.2.4	Exercice 1	6
0.3	__init__, Attributs d'Instance vs de Classe	6
0.3.1	Méthode __init__	6
0.3.2	Exemple : Classe CompteBancaire	6
0.3.3	Attributs d'Instance vs de Classe	6
0.3.4	Exemple : Attribut de Classe	7
0.3.5	Exercice 2	7
0.4	Encapsulation et Attributs Privés	7
0.4.1	Principe d'Encapsulation	7
0.4.2	Exemple : Attributs Privés	7
0.4.3	Getters et Setters	7
0.4.4	Diagramme UML : Classe CompteBancaire	8
0.5	Méthodes Spéciales et Validation	8
0.5.1	Méthodes Spéciales	8
0.5.2	Exemple : Redéfinir __repr__ et __eq__	8
0.5.3	Validation dans les Setters	9
0.6	Polymorphisme et Classes Abstraites	9
0.6.1	Polymorphisme	9
0.6.2	Diagramme UML : Polymorphisme	9
0.6.3	Classes Abstraites	10
0.7	Listes d'Objets Hétérogènes	10
0.8	Bonnes Pratiques	11

0.8.1	Clean Code	11
0.8.2	Documentation avec des Docstrings	11
0.9	Exercices Supplémentaires	12
0.9.1	Exercice 3 : Classe <code>Livre</code>	12
0.9.2	Exercice 4 : Classe Abstraite <code>Animal</code>	12
	Résumé des Concepts Clés	13
	Ressources pour Aller Plus Loin	13

0.1 Introduction et Rappels sur Python

0.1.1 Pourquoi la POO ?

La programmation orientée objet (POO) apporte de nombreux avantages pour structurer les projets logiciels :

- **Modularité** : le code est organisé en classes réutilisables.
- **Encapsulation** : les données internes sont protégées.
- **Polymorphisme** : flexibilité dans l'implémentation des méthodes.
- **Héritage** : réutilisation de code via des classes parentes.

0.1.2 Rappel : Types de Base en Python

Python dispose d'un ensemble de types primitifs couramment utilisés : `int`, `float`, `str`, `list`, `dict`, `tuple`, `set`.

```
1 age = 25          # int
2 nom = "Alice"     # str
3 notes = [12, 15]  # list
```

Listing 1 – Exemples de types de base

0.1.3 Rappel : Fonctions

```
1 def calculer_moyenne(notes):
2     return sum(notes) / len(notes)
```

Listing 2 – Définition d'une fonction

0.1.4 Rappel : Portée des Variables

- **Locale** : variable définie à l'intérieur d'une fonction.
- **Globale** : variable accessible dans tout le fichier.
- **Nonlocal** : variable accessible dans une fonction imbriquée.

0.1.5 Transition : Limites de la Programmation Procédurale

Pour des projets complexes, la programmation procédurale montre rapidement ses limites :

- Difficile à maintenir et à faire évoluer.

- Peu de réutilisabilité du code.
- Absence de structure claire pour modéliser la réalité.

Ces limitations motivent l'adoption de la POO.

0.2 Concepts de Base de la POO

0.2.1 Classe vs Objet

- **Classe** : un modèle (moule) décrivant une entité, ex. `Voiture`.
- **Objet** : une instance de la classe, ex. `ma_voiture = Voiture()`.

0.2.2 Exemple : Classe Voiture

```
1 class Voiture:
2     def __init__(self, couleur, modele):
3         self.couleur = couleur
4         self.modele = modele
5
6     def demarrer(self):
7         print(f"{self.modele} démarre !")
```

Listing 3 – Définition de la classe Voiture

0.2.3 Diagramme UML : Classe Voiture

```
+-----+
| Voiture |
+-----+
| - couleur: str |
| - modele: str |
+-----+
| + demarrer() |
+-----+
```

0.2.4 Exercice 1

Exercice 1 : Classe Personne

Créez une classe `Personne` avec :

- Attributs : `nom`, `age`.
- Méthode : `se_presenter()`.

Correction : Classe Personne

```
1 class Personne:
2     def __init__(self, nom, age):
3         self.nom = nom
4         self.age = age
5
6     def se_presenter(self):
7         print(f"Je m'appelle {self.nom} et j'ai {self.age}
            ans.")
```

0.3 `__init__`, Attributs d'Instance vs de Classe

0.3.1 Méthode `__init__`

La méthode `__init__` est le **constructeur** de la classe. Elle est appelée automatiquement lors de la création d'un objet et permet d'initialiser ses attributs d'instance.

0.3.2 Exemple : Classe `CompteBancaire`

```
1 class CompteBancaire:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
```

Listing 4 – Constructeur de `CompteBancaire`

0.3.3 Attributs d'Instance vs de Classe

- **Attribut d'instance** : propre à chaque objet (ex. `self.solde`).
- **Attribut de classe** : partagé par tous les objets de la classe (ex. `CompteBancaire.compteur`).

0.3.4 Exemple : Attribut de Classe

```
1 class CompteBancaire:
2     compteur = 0 # Attribut de classe
3
4     def __init__(self, titulaire):
5         self.titulaire = titulaire
6         CompteBancaire.compteur += 1
```

Listing 5 – Attribut de classe : compteur d’instances

0.3.5 Exercice 2

Exercice 2 : Comptage de comptes

Ajoutez un attribut de classe `nombre_de_comptes` à `CompteBancaire` qui s’incrémente à chaque création d’un nouveau compte.

0.4 Encapsulation et Attributs Privés

0.4.1 Principe d’Encapsulation

L’encapsulation consiste à protéger les données internes d’un objet. En Python, la convention est d’utiliser un underscore `_` comme préfixe pour indiquer qu’un attribut est « privé » (usage interne recommandé).

0.4.2 Exemple : Attributs Privés

```
1 class CompteBancaire:
2     def __init__(self, titulaire, solde):
3         self.titulaire = titulaire
4         self._solde = solde # Attribut privé
```

Listing 6 – Attribut privé avec convention underscore

0.4.3 Getters et Setters

Pour accéder ou modifier un attribut privé de façon contrôlée, on utilise des méthodes dédiées :

```
1 class CompteBancaire:
2     def get_solde(self):
```

```

3         return self._solde
4
5     def set_solde(self, montant):
6         if montant >= 0:
7             self._solde = montant

```

Listing 7 – Getter et setter du solde

0.4.4 Diagramme UML : Classe CompteBancaire

```

+-----+
| CompteBancaire |
+-----+
| - _solde: float |
| - titulaire: str |
+-----+
| + get_solde(): float |
| + set_solde(montant) |
+-----+

```

0.5 Méthodes Spéciales et Validation

0.5.1 Méthodes Spéciales

Python propose des méthodes dites « magiques » (dunder methods) pour personnaliser le comportement des objets :

- `__repr__` : représentation officielle de l'objet (debug).
- `__str__` : représentation lisible pour l'affichage.
- `__eq__` : comparaison entre deux objets.

0.5.2 Exemple : Redéfinir `__repr__` et `__eq__`

```

1 class Personne:
2     def __repr__(self):
3         return f"Personne(nom={self.nom}, age={self.age})"
4
5     def __eq__(self, autre):
6         return self.nom == autre.nom and self.age == autre.age

```


Listing 8 – Méthodes spéciales dans la classe Personne

0.5.3 Validation dans les Setters

```

1 def set_age(self, age):
2     if age > 0:
3         self.age = age
4     else:
5         raise ValueError("L'age doit etre positif.")

```

Listing 9 – Validation de l'âge dans un setter

Remarque

Il est recommandé de toujours valider les données dans les setters afin de garantir la cohérence interne de l'objet.

0.6 Polymorphisme et Classes Abstraites

0.6.1 Polymorphisme

Le polymorphisme permet à des objets de classes différentes de partager la même interface tout en ayant des comportements distincts.

Exemple : la méthode `aire()` est définie pour `Carre` et `Cercle`, mais son calcul diffère.

0.6.2 Diagramme UML : Polymorphisme

```

+-----+ +-----+
|  Forme  | | Carre  |
+-----+ +-----+
| + aire(): float |<- | + cote: float |
+-----+ | + aire(): float |
+-----+

+-----+
| Cercle |
+-----+
| - rayon: float |

```

```
| + aire(): float |
+-----+
```

0.6.3 Classes Abstraites

Une classe abstraite définit une interface sans fournir d'implémentation. Elle force les classes dérivées à implémenter certaines méthodes.

```
1 from abc import ABC, abstractmethod
2
3 class Forme(ABC):
4     @abstractmethod
5     def aire(self):
6         pass
```

Listing 10 – Classe abstraite Forme

```
1 import math
2
3 class Carre(Forme):
4     def __init__(self, cote):
5         self.cote = cote
6
7     def aire(self):
8         return self.cote ** 2
9
10 class Cercle(Forme):
11     def __init__(self, rayon):
12         self.rayon = rayon
13
14     def aire(self):
15         return math.pi * self.rayon ** 2
```

Listing 11 – Implémentations concrètes

0.7 Listes d'Objets Hétérogènes

Grâce au polymorphisme, il est possible de stocker des objets de types différents dans une même liste et d'appeler une méthode commune sur chacun d'eux :

```
1 formes = [Carre(5), Cercle(3)]
2 for forme in formes:
3     print(forme.aire())
```

Listing 12 – Itération sur une liste hétérogène

Ce mécanisme simplifie grandement l'écriture d'algorithmes génériques qui n'ont pas besoin de connaître le type exact de chaque objet.

0.8 Bonnes Pratiques

0.8.1 Clean Code

- **Lisibilité** : utiliser des noms de variables et de méthodes clairs et descriptifs.
- **Modularité** : garder les classes courtes et focalisées sur une seule responsabilité (*Single Responsibility Principle*).
- **Commentaires** : expliquer le *pourquoi*, pas le *comment* (le code bien écrit se suffit à lui-même).

0.8.2 Documentation avec des Docstrings

En Python, les docstrings permettent de documenter les classes et méthodes de façon standardisée :

```
1 def se_prresenter(self):  
2     """Affiche une présentation de la personne."""  
3     print(f"Je m'appelle {self.nom}.")
```

Listing 13 – Exemple de docstring

```
1 class CompteBancaire:  
2     """  
3     Représente un compte bancaire.  
4  
5     Attributes:  
6         titulaire (str): Nom du titulaire du compte.  
7         _solde (float): Solde courant du compte.  
8     """  
9  
10    def deposer(self, montant):  
11        """  
12        Dépose un montant sur le compte.  
13  
14        Args:
```

```
15         montant (float): Montant à déposer (doit être
16             positif).
17
18     Raises:
19         ValueError: Si le montant est négatif ou nul.
20     """
21     if montant <= 0:
22         raise ValueError("Le montant doit être positif.")
23     self._solde += montant
```

Listing 14 – Docstring complète d’une classe

0.9 Exercices Supplémentaires

0.9.1 Exercice 3 : Classe Livre

Exercice 3 : Classe Livre

Créez une classe `Livre` avec :

- Attributs : `titre`, `auteur`, `_nombre_de_pages`.
- Méthodes : `get_nombre_de_pages()`, `set_nombre_de_pages()`.

Assurez-vous que le nombre de pages est toujours un entier positif.

0.9.2 Exercice 4 : Classe Abstraite Animal

Exercice 4 : Classe abstraite Animal

Créez une classe abstraite `Animal` avec :

- Une méthode abstraite : `crier()`.
- Des classes dérivées : `Chien` et `Chat`.

Chaque animal doit afficher son cri caractéristique.

Correction : Classe Animal

```
1 from abc import ABC, abstractmethod
2
3 class Animal(ABC):
4     @abstractmethod
5     def crier(self):
6         pass
```

```
7
8 class Chien(Animal):
9     def crier(self):
10         print("Woof!")
11
12 class Chat(Animal):
13     def crier(self):
14         print("Miaou!")
15
16 # Utilisation
17 animaux = [Chien(), Chat()]
18 for animal in animaux:
19     animal.crier()
```

0.9 Résumé des Concepts Clés

Concept	Description
Classe / Objet	Modèle (moule) et instance concrète de ce modèle.
Encapsulation	Protection des données internes via des attributs privés et des accesseurs.
Polymorphisme	Interface commune, comportements différents selon le type réel de l'objet.
Héritage	Réutilisation et extension du code via des classes parentes.
Classes abstraites	Contrat d'interface que les sous-classes doivent respecter.

0.9 Ressources pour Aller Plus Loin

- **Documentation officielle Python** : <https://docs.python.org/3/>
- **Livre** : *Clean Code* — Robert C. Martin
- **Outil UML** : [Lucidchart](#)

Bonne pratique : relisez et testez chaque exercice avant de consulter la correction.